

VULNERABILITY ASSESSMENT REPORT

Target Application: Juice Shop

<https://juice-shop.herokuapp.com>

Prepared By:

Arun Kumar M

B.E. Computer Science and Engineering (Cyber Security)

SRM Madurai College for Engineering and Technology

Assessment Date: April 2026

Table of Contents

Executive Summary	3
Key Findings.....	3
Assessment Scope and Methodology	3
Target Information	3
Methodology	3
Tools and Technologies	4
Scope Limitations	4
Risk Classification Framework	4
Network Exposure Assessment	5
Detailed Vulnerability Findings	5
Medium Severity Findings	6
Finding 1: Content Security Policy (CSP) Not Implemented.....	5-6
Finding 2: Cross-Origin Resource Sharing (CORS) Misconfiguration	6-7
Finding 3: HTTP Strict Transport Security (HSTS) Not Enforced	7
Low Severity Findings	7
Finding 4: Information Disclosure via HTML/JavaScript Comments.....	7-8
Finding 5: Server Timestamp Information Disclosure.....	8
Finding 6: Cache-Control Header Misconfiguration.....	8-9
Informational Findings	9
Finding 7: Modern Web Application Framework Detected	9
Recommendations and Remediation Roadmap	10
Immediate Actions (Medium Priority).....	10
Short-term Improvements (Low Priority)	10
Long-term Security Enhancements	10
Conclusion	10-11
Next Steps	10-11
Appendix.....	11
A. Testing Methodology Detail.....	11-12
B. Reference Information	11-12
C. Disclaimer.....	11-12

Executive Summary

This report presents the findings of a comprehensive security assessment conducted on the publicly accessible web application Juice Shop (<https://juice-shop.herokuapp.com>). The primary objective was to identify common security misconfigurations and vulnerabilities using non-intrusive, passive reconnaissance techniques.

The assessment was limited to publicly available resources and adhered strictly to ethical testing boundaries. No exploitation attempts were performed, and all testing was conducted within safe and legally permitted parameters.

Key Findings

- **No high-severity vulnerabilities identified**
- 3 medium-severity issues requiring attention
- 3 low-severity issues with minimal impact
- 3 informational findings for awareness
- Primary concerns: Missing security headers (CSP, HSTS) and CORS misconfiguration

Overall, the application demonstrates a moderate security posture with room for improvement through implementation of industry-standard security controls.

Assessment Scope and Methodology

Target Information

Attribute	Details
Application Name	Juice Shop
URL	https://juice-shop.herokuapp.com
IP Address	46.137.15.86
Assessment Type	Passive Security Review (Read-Only)
Assessment Date	April 2026
Assessor	Arun Kumar M, B.E. CSE (Cyber Security)

Methodology

The assessment followed a structured approach consisting of four primary phases:

- **Reconnaissance:** Information gathering and network mapping
- **Passive Scanning:** Automated vulnerability detection using OWASP ZAP
- **Manual Verification:** Browser-based inspection of security headers and configurations

- **Reporting: Documentation and risk classification**

Tools and Technologies

Tool	Purpose
Nmap	Network port scanning and service enumeration
OWASP ZAP	Passive web application vulnerability scanning
Browser DevTools	Security header inspection and cookie analysis

Scope Limitations

The following limitations apply to this assessment:

- No active exploitation or penetration testing was performed
- Assessment limited to publicly accessible web pages only
- No authenticated testing or privilege escalation attempts
- Cloud hosting infrastructure may affect certain findings
- Results reflect the application state at the time of testing
-

Risk Classification Framework

All identified vulnerabilities have been categorized using a four-tier risk classification system based on exploitability, potential impact, and likelihood of occurrence.

Risk Level	Description	Count
High	Easily exploitable vulnerabilities with severe security impact requiring immediate remediation	0
Medium	Moderate vulnerabilities that may require specific conditions to exploit but should be addressed	3
Low	Minor security issues with limited impact on overall security posture	3
Informational	Observations with no direct security threat but may aid attackers in reconnaissance	3

Network Exposure Assessment

Network reconnaissance was performed using Nmap to identify exposed services and potential attack vectors. The following open ports were detected on the target system:

Port	Service	Notes
21	FTP	File transfer protocol
25	SMTP	Email transmission
80	HTTP	Web server (primary)
110	POP3	Email retrieval
143	IMAP	Email access
443	HTTPS	Secure web server
445	Microsoft-DS	SMB file sharing
2000	Cisco SCCP	Cisco protocol
5060	SIP	VoIP signaling
8010	HTTP Proxy	Web proxy service

Assessment Notes:

- Multiple services exposed may be due to cloud hosting infrastructure
- Primary web services operating on standard ports 80 and 443
- No evidence of unnecessary services running on non-standard ports
-

Detailed Vulnerability Findings

The following sections detail all identified vulnerabilities, organized by severity level. Each finding includes technical evidence, potential security impact, and recommended remediation steps.

Medium Severity Findings

Finding 1: Content Security Policy (CSP) Not Implemented

Severity	Medium
Affected Ports	80 / 443

Description:

The application does not implement a Content Security Policy, which is a critical security header that controls the sources from which content can be loaded and executed in the browser. This header acts as an additional layer of defense against Cross-Site Scripting (XSS) and data injection attacks.

Evidence:

Analysis of HTTP response headers revealed the complete absence of the Content-Security-Policy header. This was confirmed through both automated scanning (OWASP ZAP) and manual verification using browser developer tools.

Security Impact:

Without CSP protection, the application is significantly more vulnerable to XSS attacks, unauthorized script execution, and malicious content injection. Attackers may be able to inject and execute arbitrary JavaScript code, steal sensitive user data, or perform actions on behalf of authenticated users.

Recommended Remediation:

Implement a strict Content Security Policy that explicitly whitelists trusted content sources. Recommended initial policy: Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self' Gradually remove 'unsafe-inline' directives as inline scripts and styles are refactored.

Finding 2: Cross-Origin Resource Sharing (CORS) Misconfiguration

Severity	Medium
Affected Ports	80 / 443

Description:

The server is configured with an overly permissive CORS policy, allowing access from all domains using the wildcard configuration 'Access-Control-Allow-Origin: *'. This misconfiguration permits any third-party website to make cross-origin requests to the application and access potentially sensitive resources.

Evidence:

HTTP response headers contain 'Access-Control-Allow-Origin: *', indicating unrestricted cross-origin access. This was observed consistently across multiple endpoints during passive scanning.

Security Impact:

Unauthorized domains can make authenticated requests to the application, potentially accessing or modifying sensitive user data. This could enable sophisticated attacks including: • Cross-site request forgery (CSRF) • Unauthorized data exfiltration • Session hijacking from malicious sites • Privacy violations through unauthorized API access

Recommended Remediation:

Replace the wildcard CORS policy with a whitelist of specific trusted domains: Access-Control-Allow-Origin: https://trusted-domain.com For applications requiring multiple trusted origins, implement server-side logic to validate the request origin against a whitelist before setting the header dynamically. Never use wildcards in production environments handling sensitive data.

Finding 3: HTTP Strict Transport Security (HSTS) Not Enforced

Severity	Medium
Affected Ports	80 / 443

Description:

The application does not enforce HTTPS-only communication through the Strict-Transport-Security header. This allows browsers to potentially connect over insecure HTTP connections, exposing user traffic to interception and manipulation.

Evidence:

The Strict-Transport-Security response header is absent from all server responses. Additionally, the application accepts connections on port 80 without automatic redirection to HTTPS.

Security Impact:

Users may inadvertently connect over unencrypted HTTP connections, especially on first visit or after clearing browser data. This exposes:

- Session cookies and authentication tokens to network sniffing
- User credentials during login
- Sensitive application data to man-in-the-middle attacks
- Protocol downgrade attacks (SSL stripping)

Recommended Remediation:

Implement HSTS with the following header: Strict-Transport-Security: max-age=31536000; include Subdomains; preload These forces browsers to use HTTPS exclusively for one year, including all subdomains. Consider submitting the domain to the HSTS preload list for maximum protection. Additionally, configure HTTP (port 80) to immediately redirect all traffic to HTTPS.

Low Severity Findings

Finding 4: Information Disclosure via HTML/JavaScript Comments

Severity	Low
Affected Ports	80 / 443

Description:

The application source code contains HTML and JavaScript comments that expose internal information about the application architecture, development practices, and implementation details.

Evidence:

Manual inspection of page source revealed multiple developer comments containing technical details about code structure, debugging information, and implementation notes. These comments are visible to any user who views the page source.

Security Impact:

While not directly exploitable, these comments may assist attackers in understanding the application structure, identifying potential attack vectors, and planning more sophisticated attacks. Information exposed includes:

- Internal file paths and directory structures
- Framework and library versions
- Developer notes and TODO items
- Debugging logic and test code

Recommended Remediation:

Remove all unnecessary comments from production code before deployment. Implement a build process that automatically strips comments from HTML, JavaScript, and CSS files. Maintain commented source code in version control only, not in production deployments.

Finding 5: Server Timestamp Information Disclosure

Severity	Low
Affected Ports	80 / 443

Description:

HTTP response headers include server timestamp information that reveals details about server time synchronization and may assist attackers in analyzing server behavior patterns.

Evidence:

Timestamp metadata was observed in various HTTP response headers, including Last-Modified and Date headers with precise timing information.

Security Impact:

Timestamp information has minimal direct security impact but can aid attackers in:

- Determining server time zone and synchronization
- Analyzing response time patterns
- Timing correlation attacks
- Reconnaissance for automated attack scheduling

Recommended Remediation:

While timestamps in Date headers are standard HTTP practice, review and minimize unnecessary timestamp metadata in custom headers. Consider implementing rate limiting and request throttling to mitigate timing analysis attacks.

Finding 6: Cache-Control Header Misconfiguration

Severity	Low
Affected Ports	80 / 443

Description:

The application uses weak or missing Cache-Control headers, potentially allowing sensitive information to be cached by browsers or intermediary proxy servers.

Evidence:

Analysis of HTTP response headers revealed absent or insufficiently restrictive Cache-Control directives on pages that may handle sensitive data.

Security Impact:

Sensitive user data, session information, or personally identifiable information (PII) may be stored in browser cache or proxy server cache, where it could be:

- Accessed by unauthorized users on shared computers
- Retrieved from proxy server logs
- Exposed through backup or forensic analysis
- Persisted longer than intended

Recommended Remediation:

Implement strict caching policies for pages handling sensitive data: Cache-Control: no-store, no-cache, must-revalidate, private Pragma: no-cache Expires: 0 For public static resources (images, CSS, JavaScript), use appropriate cache directives with version control to allow safe caching while maintaining update capability.

Informational Findings

Finding 7: Modern Web Application Framework Detected

Severity	Informational
Affected Ports	80 / 443

Description:

The application utilizes modern JavaScript frameworks and libraries, as detected through passive analysis of client-side code and HTTP headers.

Evidence:

Framework signatures were identified during OWASP ZAP scanning and manual source code review, indicating the use of contemporary web development technologies.

Security Impact:

This finding carries no direct security risk. Modern frameworks typically include built-in security features and regular security updates. However, ensure:

- All frameworks and libraries are kept up to date
- Known vulnerabilities in dependencies are monitored
- Security best practices for the specific framework are followed

Recommended Remediation:

No immediate action required. Maintain a regular update schedule for all JavaScript frameworks and dependencies. Implement automated dependency scanning to identify vulnerable components.

Recommendations and Remediation Roadmap

The following recommendations are prioritized based on risk severity and ease of implementation. Addressing these items will significantly improve the application's security posture.

Immediate Actions (Medium Priority)

These items should be addressed within the next 30 days:

- **Implement Content Security Policy (CSP)**

Deploy a strict CSP header to mitigate XSS risks. Start with a permissive policy and gradually tighten based on application requirements.

- **Restrict CORS Policy**

Replace wildcard CORS configuration with explicit whitelist of trusted domains.

- **Enable HTTP Strict Transport Security (HSTS)**

Configure HSTS header with appropriate max-age value and consider HSTS preload list submission.

Short-term Improvements (Low Priority)

These items should be addressed within the next 90 days:

- Remove all unnecessary comments from production code
- Implement secure cache control headers for sensitive pages
- Review and minimize timestamp disclosure in HTTP headers
- Establish automated dependency scanning for JavaScript frameworks

Long-term Security Enhancements

- Implement Web Application Firewall (WAF) for additional protection
- Conduct regular automated security scanning as part of CI/CD pipeline
- Perform periodic penetration testing with authorized exploitation attempts
- Establish security awareness training program for development team
- Implement comprehensive security logging and monitoring

Conclusion

This passive security assessment of the Juice Shop web application has identified several security configuration issues that, while not representing critical vulnerabilities, warrant attention and remediation. The absence of high-severity findings is encouraging and suggests that the application has reasonable baseline security controls in place.

The primary areas requiring improvement involve the implementation of modern security headers (Content Security Policy and HTTP Strict Transport Security) and the correction of CORS misconfiguration. These are well-understood security controls with established best practices and should be straightforward to implement.

The low-severity and informational findings represent minor security hygiene issues that should be addressed as part of routine security maintenance. While individually they present minimal risk, collectively they contribute to a more secure application posture.

Overall Risk Assessment: The application demonstrates a moderate security posture with no critical vulnerabilities identified through passive testing. Implementation of the recommended security controls will significantly enhance the application's defense-in-depth capabilities and align it more closely with industry security standards.

Next Steps

- Review and prioritize remediation efforts based on risk classification
- Assign ownership and timelines for addressing each finding
- Consider conducting active penetration testing to identify additional vulnerabilities
- Establish regular security assessment schedule (recommended: quarterly)
- Integrate security testing into development lifecycle and CI/CD pipeline

Appendix

A. Testing Methodology Detail

Network Scanning Approach:

Nmap was configured for comprehensive TCP port scanning across common service ports (1-10000). The scan utilized service version detection (-sV) to identify running services and their versions. OS detection was not performed to maintain passive posture.

Web Application Scanning Approach:

OWASP ZAP was configured in passive scanning mode, analyzing all HTTP traffic without sending active attack payloads. The spider was limited to prevent excessive requests. All findings were manually verified using browser developer tools to confirm validity.

B. Reference Information

Industry Standards and Guidelines:

- OWASP Top 10 Web Application Security Risks
- NIST SP 800-115: Technical Guide to Information Security Testing
- CWE/SANS Top 25 Most Dangerous Software Errors
- OWASP Application Security Verification Standard (ASVS)

C. Disclaimer

This assessment was conducted for educational purposes as part of academic coursework in cybersecurity. The findings represent vulnerabilities identified at a specific point in time using passive testing methodologies. Active exploitation and penetration testing were not performed.

The information contained in this report should be used solely for legitimate security improvement purposes. All testing was conducted within ethical and legal boundaries, limiting activities to passive observation of publicly accessible resources.

Results are accurate to the best of the assessor's ability using the stated methodology and tools. However, no security assessment can guarantee the identification of all vulnerabilities. Regular ongoing security testing is recommended.

```
kali@LAPTOP-V7JICG3L: ~  
WARNING: No targets were specified, so 0 hosts scanned.  
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.15 seconds  
  
(kali@LAPTOP-V7JICG3L)-[~]  
$ sudo service mariadb start  
Failed to start mariadb.service: Unit mariadb.service not found.  
  
(kali@LAPTOP-V7JICG3L)-[~]  
$ nmap -sV juice-shop.herokuapp.com  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-14 18:59 +0530  
Stats: 0:00:57 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan  
Service scan Timing: About 10.00% done; ETC: 19:04 (0:04:21 remaining)  
Stats: 0:00:59 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan  
Service scan Timing: About 10.00% done; ETC: 19:04 (0:04:30 remaining)  
Nmap scan report for juice-shop.herokuapp.com (46.137.15.86)  
Host is up (0.12s latency).  
Other addresses for juice-shop.herokuapp.com (not scanned): 54.73.53.134 54.220.192.176  
rDNS record for 46.137.15.86: ec2-46-137-15-86.eu-west-1.compute.amazonaws.com  
Not shown: 989 filtered tcp ports (no-response)  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp?  
25/tcp    open  smtp?  
80/tcp    open  http          Heroku  
110/tcp   open  pop3?  
113/tcp   closed ident  
143/tcp   open  imap?  
443/tcp   open  ssl/https     heroku-router  
445/tcp   open  microsoft-ds?  
2000/tcp  open  cisco-sccp?  
5060/tcp  open  sip?  
8010/tcp  open  ssl/http-proxy FortiGuard Web Filtering
```

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)**
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)
- Information Disclosure - Suspicious Comments (3)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Cross-Domain Misconfiguration

URL: <https://juice-shop.herokuapp.com/chunk-24EZL24.js>

Risk: Medium

Confidence: Medium

Parameter:

Attack: Access-Control-Allow-Origin: *

Evidence: CWE ID: 264 WASC ID: 14

Source: Passive (10098 - Cross-Domain Misconfiguration)

Input Vector:

Description: Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Other info: The CORS misconfiguration on the web server permits cross-domain read requests from arbitrary third party domains, using unauthenticated APIs on this domain. Web browser implementations do not permit arbitrary third parties to read the response from authenticated APIs, however. This reduces the risk somewhat. This misconfiguration could be used by an attacker to access data that is available in an unauthenticated manner, but which uses some other form of security, such as IP address white-listing.

Solution: Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance). Configure the 'Access-Control-Allow-Origin' HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

Reference: <https://vuln.catfortify.com/en/detail?category=HTML5&subcategory=Overly%20Permissive%20CORS%20Policy>

Alert Tags:

Key	Value
OWASP_2021_A01	https://owasp.org/Top10/A01_2021-Broken_Access_Control/
OWASP_2017_A05	https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control.html
CWE-264	https://cwe.mitre.org/data/definitions/264.html
POLICY_QA_STD	
POLICY_PENTEST	

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)**
- Timestamp Disclosure - Unix (Systemic)
- Information Disclosure - Suspicious Comments (3)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Strict-Transport-Security Header Not Set

URL: <https://juice-shop.herokuapp.com/assets/public/favicon.ico>

Risk: Low

Confidence: High

Parameter:

Attack:

Evidence: CWE ID: 319 WASC ID: 15

Source: Passive (10035 - Strict-Transport-Security Header)

Alert Reference: 10035-1

Input Vector:

Description: HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL). HSTS is an IETF standards track protocol and is specified in RFC 6797.

Other info:

Solution: Ensure that your web server, application server, load balancer, etc. is configured to enforce Strict-Transport-Security.

Reference: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html
<https://owasp.org/www-community/Security-Headers>
https://en.wikipedia.org/wiki/HTTP_Strict_Transport_Security

Alert Tags:

Key	Value
OWASP_2021_A05	https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
OWASP_2017_A06	https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration.html
POLICY_QA_STD	

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)**
- Information Disclosure - Suspicious Comments (3)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Timestamp Disclosure - Unix

URL: <https://juice-shop.herokuapp.com/assets/public/favicon.ico>

Risk: Low

Confidence: Low

Parameter: Reporting-Endpoints

Attack:

Evidence: 1776174795

CWE ID: 497

WASC ID: 13

Source: Passive (10096 - Timestamp Disclosure)

Input Vector:

Description:

A timestamp was disclosed by the application/web server. - Unix

Other Info:

1776174795, which evaluates to: 2026-04-14 19:23:15.

Solution:

Manually confirm that the timestamp data is not sensitive, and that the data cannot be aggregated to disclose exploitable patterns.

Reference:

<https://cwe.mitre.org/data/definitions/260.html>

Alert Tags:

Key	Value
OWASP_2021_A01	https://owasp.org/Top10/A01_2021-Broken_Access_Control/
OWASP_2017_A03	https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure.html
POLICY_PENTEST	https://cwe.mitre.org/data/definitions/497.html
CWE-497	https://www.zaproxy.org/docs/desktop/addons/common-library/alerttags/#systemic
SYSTEMIC	

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)
- Information Disclosure - Suspicious Comments (3)**
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Information Disclosure - Suspicious Comments

URL: <https://juice-shop.herokuapp.com/chunk-LHK57QUN.js>

Risk: Informational

Confidence: Low

Parameter:

Attack:

Evidence: db

CWE ID: 615

WASC ID: 13

Source: Passive (10027 - Information Disclosure - Suspicious Comments)

Input Vector:

Description:

The response appears to contain suspicious comments which may help an attacker.

Other Info:

The following pattern was used: `{b0B}` and was detected in likely comment: `"/angular.dev/best-practices/security#preventing-cross-site-scripting-xss"-v=class extends Error(code:constructor.t.n){super(ve*}; see evidence field for the suspicious comment/snippet.`

Solution:

Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.

Reference:

Alert Tags:

Key	Value
OWASP_2021_A01	https://owasp.org/Top10/A01_2021-Broken_Access_Control/
WSTG-42-INFO-05	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_...
OWASP_2017_A03	https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure.html
POLICY_PENTEST	
CWE-615	https://cwe.mitre.org/data/definitions/615.html

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode Zest Results WebSockets Requester AJAX Spider Rate Limit Automation Progress Fuzzer Client Map Client History Client Spider Spider OAST Insights Auth Diags Output

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)
- Information Disclosure - Suspicious Comments (3)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Modern Web Application

URL: <https://juice-shop.herokuapp.com/itemap.xml>

Risk: Informational

Confidence: Medium

Parameter:

Attack:

```
<script>
window.addEventListener('load', function(){
  window.cookieconsent.initialise({
    'palette': {
      'popup': { 'background': 'var(--theme-primary)', 'text': 'var(--theme-text)' },
      'button': { 'background': 'var(--theme-accent)', 'text': 'var(--theme-text)' }
    },
    'theme': 'classic',
    'position': 'bottom-right',
    'content': { 'message': 'This website uses fruit cookies to ensure you get the juiciest tracking experience.', 'dismiss': 'Me want it!', 'link': 'But me wait!', 'href': 'https://www.youtube.com/watch?v=9PnbKL3wU44' }
  });
});
</script>
```

Evidence:

CWE ID: 525

WASC ID: 13

Source: Passive (10109 - Modern Web Application)

Input Vector:

Description:

The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.

Other info:

No links have been found while there are scripts, which is an indication that this is a modern web application.

Solution:

This is an informational alert and so no changes are required.

Reference:

Alert Tags:

Key	Value
POLICY_QA_STD	
POLICY_PENTEST	
SYSTEMIC	https://www.zaproxy.org/docs/desktop/addons/common-library/alerttags/#systemic
POLICY_DEV_STD	

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode Zest Results WebSockets Requester AJAX Spider Rate Limit Automation Progress Fuzzer Client Map Client History Client Spider Spider OAST Insights Auth Diags Output

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)
- Information Disclosure - Suspicious Comments (3)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Re-examine Cache-control Directives

URL: <https://juice-shop.herokuapp.com/robots.txt>

Risk: Informational

Confidence: Low

Parameter: cache-control

Attack:

Evidence:

CWE ID: 525

WASC ID: 13

Source: Passive (10015 - Re-examine Cache-control Directives)

Input Vector:

Description:

The cache-control header has not been set properly or is missing, allowing the browser and proxies to cache content. For static assets like css, js, or image files this might be intended, however, the resources should be reviewed to ensure that no sensitive content will be cached.

Other info:

Solution:

For secure content, ensure the cache-control HTTP header is set with 'no-cache, no-store, must-revalidate'. If an asset should be cached consider setting the directives 'public, max-age, immutable'.

Reference:

- https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html#web-content-caching
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/headers/Cache-Control>
- [https://grayduck.mn\(2021/09/13/cache-control-recommendations/](https://grayduck.mn(2021/09/13/cache-control-recommendations/)

Alert Tags:

Key	Value
WSTG-V42-ATHN-06	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/04-Authentic...
CWE-525	https://cwe.mitre.org/data/definitions/525.html
POLICY_PENTEST	
SYSTEMIC	https://www.zaproxy.org/docs/desktop/addons/common-library/alerttags/#systemic

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Test Results WebSockets Requester AJAX Spider Rate Limit Automation Progress Fuzzer Client Map Client History Client Spider Spider OAST Insights Auth Diags Output

Quick Start Request Response Sites History Search Break Breakpoints Alerts Active Scan Forced Browse Params Scripts HTTP Sessions

Alerts (7)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)
- Information Disclosure - Suspicious Comments (3)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)

Content Security Policy (CSP) Header Not Set

URL: <https://juice-shop.herokuapp.com/itemap.xml>

Risk: Medium

Confidence: High

Parameter:

Attack:

Evidence:

CWE ID: 693

WASC ID: 15

Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)

Alert Reference: 10038-1

Input Vector:

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as java applets, ActiveX, audio and video files.

Other Info:

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Reference:

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>
- https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- <https://www.w3.org/TR/CSP/>

Alert Tags:

Key	Value
CWE-693	https://cwe.mitre.org/data/definitions/693.html
OWASP_2021_A05	https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
OWASP_2017_A06	https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration.html
POLICY_QA_STD	
POLICY_PENTEST	

END OF REPORT